Manobi Sentinel — Documentacion

Manobi Sentinel 4
1. Requisitos del servidor 4
2. Instalación de Docker en Debian 12 (desde cero) 4
3. Ajustes del kernel (Redis + PostgreSQL) 5
4. Preparar directorios de datos 5
5. Clonar/copiar el proyecto y configurar `.env` 5
6. Certificados SSL 6
7. Descargar tiles base de Colombia (on-premise) 6
8. Levantar la plataforma 6
9. Backups automáticos 7
10. Operación básica 7
11. Estructura del proyecto 7
12. Roadmap de sesiones 7
13. Troubleshooting para operadores 8
Propuesta de valor — Manobi Sentinel 9
Sistema de alerta temprana climática para Parques Nacionales Naturales de Colombia 9
Resumen ejecutivo 9
Métricas de operación (primeras 48h) 9
Capacidades vs. estado anterior 9
Arquitectura técnica 10
Costos operativos estimados 10
Propuesta de servicios BC Fabric SAS 11
Contacto 11
Arquitectura — Manobi Sentinel 12
Diagrama de arquitectura (Mermaid) 12
Vista general (ASCII fallback) 15
Redes Docker 15
Decisiones clave 15
Flujo de alerta (Mermaid) 16
Flujo de alerta (ASCII fallback) 17
Manual de operación — Manobi Sentinel 18
1. Tareas rutinarias 18
2. Servicios y puertos internos 18

3. Health checks y URLs 19
4. Backup y restauración 19
5. Cargar shapefile oficial PNN (RUNAP) 19
6. Rotar certificado SSL 20
7. Rotar secretos 21
8. IDEAM 21
9. Motor de alertas 21
10. Modelos IA (ai-service) 22
11. Grafana alerting 22
12. Troubleshooting rápido 23
13. Checklist post-instalación 23
Manual de usuario — Manobi Sentinel 24
Sistema de alerta temprana climática para PNN Colombia 24
Guía de presentación — Manobi Sentinel 27
Para revisión/entrega a producción (PNN Colombia) 27
1. Checklist 24 horas antes 27
2. Elevator pitch (60 segundos) 28
3. Script de la presentación (30–40 min) 28
4. Demo paso a paso (cómo hacerlo en vivo) 29
5. Preguntas frecuentes (y cómo responder) 30
6. Qué dejas instalado para el equipo receptor 31
7. Checklist final antes de entrar a la reunión 32
8. Si algo falla durante el demo 32
Configuración SMTP y CI/CD 34
SMTP Relay (#1) 34
CI/CD con GitHub (#3) 35
Guía de validación UX — Manobi Sentinel 36
Requisitos 36
Flujo de validación 36
Feedback del operador 38

Manobi Sentinel

Sistema de alerta temprana climatica para Parques Nacionales Naturales de Colombia

Documentacion completa del proyecto
Generado: 2026-04-17
Repositorio: github.com/fbolivar/manobi-sentinel

Manobi Sentinel

Plataforma web **on-premise** de alerta temprana climática para Parques Nacionales Naturales de Colombia. 100% Open Source, sin dependencias de servicios cloud de pago, desplegada en Debian 12 con Docker.

> **Estado actual:** Sesión 1 — scaffolding inicial. Ver `docs/` para roadmap de sesiones siguientes (backend NestJS, frontend React, IA, IDEAM, reportes).

---

1. Requisitos del servidor

- **OS:** Debian 12 (Bookworm) — probado en IP `192.168.50.5`.
- **Docker:** 24+ y Docker Compose v2.
- **CPU/RAM mínimos:** 4 vCPU / 8 GB RAM (recomendado 8 vCPU / 16 GB).
- **Discos montados:**
  - `/opt/manobi/postgres`
  - `/opt/manobi/minio`
  - `/opt/manobi/tiles`
  - `/opt/manobi/monitoring`
- **Puertos expuestos:** 22, 80, 443.

2. Instalación de Docker en Debian 12 (desde cero)

```
 sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg lsb-release

sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | \
    sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
        \
  https://download.docker.com/linux/debian bookworm stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
        docker-compose-plugin
sudo usermod -aG docker "$USER"
newgrp docker
docker --version && docker compose version
```

3. Ajustes del kernel (Redis + PostgreSQL)

```
# Redis (evita "Background save may fail under low memory")
echo 'vm.overcommit_memory = 1' | sudo tee -a /etc/sysctl.d/99-manobi.conf

# PostgreSQL / Docker — file descriptors y conexiones
cat <<EOF | sudo tee -a /etc/sysctl.d/99-manobi.conf
fs.file-max = 2097152
net.core.somaxconn = 4096
net.ipv4.tcp_max_syn_backlog = 4096
vm.swappiness = 10
EOF
sudo sysctl --system

# ulimits para Docker
sudo mkdir -p /etc/systemd/system/docker.service.d
cat <<EOF | sudo tee /etc/systemd/system/docker.service.d/override.conf
[Service]
LimitNOFILE=1048576
EOF
sudo systemctl daemon-reload && sudo systemctl restart docker
```

4. Preparar directorios de datos

```
sudo mkdir -p /opt/manobi/{postgres,minio,tiles,monitoring,backups}
sudo chown -R $USER:$USER /opt/manobi
```

5. Clonar/copiar el proyecto y configurar `.env`

```
cd /opt
sudo mkdir -p manobi-sentinel && sudo chown $USER:$USER manobi-sentinel
# Copia el contenido del repo aquí, luego:
cd manobi-sentinel
cp .env.example .env
# Edita .env y reemplaza TODAS las cadenas CHANGE_ME con contraseñas fuertes.
# Genera secretos JWT con: openssl rand -hex 48
```

6. Certificados SSL

Opción A — Autofirmado (red interna, sin internet)

```
openssl req -x509 -nodes -days 3650 -newkey rsa:4096 \
  -keyout nginx/ssl/manobi.key \
  -out nginx/ssl/manobi.crt \
  -subj "/C=CO/ST=Cundinamarca/L=Bogota/O=PNN/OU=Manobi/CN=192.168.50.5" \
  -addext "subjectAltName=IP:192.168.50.5,DNS:sentinel.parques.gov.co"
chmod 600 nginx/ssl/manobi.key
```

Opción B — Let's Encrypt (si hay salida a internet y dominio público)

Usar Certbot con el plugin standalone antes de levantar Nginx, y luego copiar los certificados a `nginx/ssl/manobi.crt` y `nginx/ssl/manobi.key`.

7. Descargar tiles base de Colombia (on-premise)

```
# Desde una máquina con internet:
wget https://download.geofabrik.de/south-america/colombia-latest.osm.pbf

# Convertir a .mbtiles con tilemaker (instrucciones: https://github.com/systemed/
        tilemaker)
sudo apt-get install -y tilemaker
tilemaker --input colombia-latest.osm.pbf --output colombia.mbtiles

# Copiar al servidor:
scp colombia.mbtiles user@192.168.50.5:/opt/manobi/tiles/
```

Si no se dispone del archivo al inicio, el servicio `tileserver-gl` seguirá arrancando y se podrá añadir el mbtiles posteriormente (reiniciar el contenedor).

8. Levantar la plataforma

```
cd /opt/manobi-sentinel
docker compose pull
docker compose build
docker compose up -d

# Verificar estado
docker compose ps
docker compose logs -f api
```

Verificación rápida:

- `https://192.168.50.5/` → frontend
- `https://192.168.50.5/api/docs` → Swagger (NestJS)
- `https://192.168.50.5/grafana/` → monitoreo

- `https://192.168.50.5/geoserver/web/` → GeoServer
- `https://192.168.50.5/tiles/` → TileServer GL

9. Backups automáticos

Agendar `database/backups/backup.sh` en el cron del host:

```
chmod +x /opt/manobi-sentinel/database/backups/backup.sh
(sudo crontab -l 2>/dev/null; echo "0 2 * * * /opt/manobi-sentinel/database/backups/
        backup.sh >> /var/log/manobi-backup.log 2>&1") | sudo crontab -
```

10. Operación básica

Acción	Comando
Ver logs de un servicio	`docker compose logs -f <servicio>`
Reiniciar un servicio	`docker compose restart <servicio>`
Reconstruir tras cambio	`docker compose up -d --build <servicio>`
Apagar todo	`docker compose down`
Apagar + limpiar vols	`docker compose down -v` (⚠ borra datos)

11. Estructura del proyecto

```
manobi-sentinel/
├── docker-compose.yml           # Orquestación completa
├── docker-compose.dev.yml       # Override desarrollo
├── .env.example                 # Variables documentadas
├── nginx/                       # Reverse proxy + SSL + headers
├── backend/                     # API NestJS (Sesión 2)
├── frontend/                    # React + OpenLayers (Sesión 3)
├── ai-service/                  # FastAPI + scikit-learn (Sesión 4)
├── database/init/               # Esquema, índices, seeds PostGIS
├── database/backups/            # Script backup host
├── tileserver-gl/               # Tiles base on-premise
├── monitoring/                  # Prometheus + Grafana + Loki
├── geoserver/                   # Data dir montado
└── docs/                        # Arquitectura, operación, API
```

12. Roadmap de sesiones

- **Sesión 2:** Backend NestJS completo (auth, CRUD parques, motor de alertas, WebSocket, auditoría).

- **Sesión 3:** Frontend React (dashboard NASA dark, mapa OpenLayers, panel alertas, responsive).
- **Sesión 4:** Microservicio IA (FastAPI, predicción incendio/inundación, heatmap).
- **Sesión 5:** Integración IDEAM (ingesta real + simulador), notificaciones email/webhook, reportes PDF/XLSX/CSV.

13. Troubleshooting para operadores

Síntoma	Acción
El frontend no carga	`docker compose logs nginx frontend`
La API responde 502	`docker compose restart api && docker compose logs -f api`
PostgreSQL no inicia	Verificar permisos de `/opt/manobi/postgres`
Redis avisa de overcommit	Revisar `vm.overcommit_memory` (paso 3)
Grafana no muestra datos	Revisar `monitoring/prometheus/prometheus.yml`
El mapa no muestra tiles	Copiar `colombia.mbtiles` a `/opt/manobi/tiles/` y reiniciar
Alertas por email no llegan	Revisar variables `SMTP_RELAY_*` en `.env`

Licencia: Open Source (pendiente definir licencia específica con PNN). **Contacto técnico:** completar antes de puesta en producción.

Propuesta de valor — Manobi Sentinel

Sistema de alerta temprana climática para Parques Nacionales Naturales de Colombia

---

Resumen ejecutivo

Manobi Sentinel es una plataforma web on-premise que monitorea en tiempo real las condiciones climáticas y riesgos de incendio en los 73 parques nacionales naturales de Colombia, utilizando datos de 8 fuentes oficiales (IDEAM + NASA + OroraTech) y modelos de inteligencia artificial predictiva.

Métricas de operación (primeras 48h)

Indicador	Valor
Parques monitoreados	**73** (100% del SINAP con geometría oficial RUNAP)
Fuentes de datos activas	**8** (5 IDEAM + VIIRS + MODIS + OroraTech)
Hotspots satelitales captados	**35,453** en 48h
Eventos meteorológicos ingestados	**110,982** en 48h
Predicciones IA generadas	**2,502**
Alertas generadas	**64** (49 rojas + 15 amarillas)
Tiempo de detección	**5 segundos** por ciclo
Frecuencia de evaluación	Cada **15 minutos**
Disponibilidad	**24/7** on-premise, sin dependencia cloud

Capacidades vs. estado anterior

Capacidad	Sin Manobi	Con Manobi
Detección de incendio forestal	Manual, reportes tardíos	Automática, satelital cada 30 min
Cobertura de parques	Parcial, según recursos de campo	73/73 parques monitoreados 24/7
Alertas	Ninguna automatizada	

Capacidad	Sin Manobi	Con Manobi
		Reglas configurables por nivel/ variable
Notificación a operadores	Llamada telefónica	Email automático + push + webhook
Datos meteorológicos	Consulta manual portal IDEAM	5 variables ingestadas automáticamente
Predicción	Ninguna	IA incendio + inundación + forecast 72h
Histórico	Archivos Excel	Dashboard filtrable + exportable CSV/PDF/XLSX
Auditoría	Ninguna	Log completo por usuario con timestamps

Arquitectura técnica

- **100% on-premise** — cumple requisitos gubernamentales de soberanía de datos
- **Sin costos de cloud** — corre en un servidor Debian con Docker
- **Open source** — sin licencias recurrentes de software
- **Certificado SSL** — comunicaciones cifradas TLS 1.2/1.3
- **Respaldo automático** — backup diario PostgreSQL + MinIO con retención 30 días

Stack tecnológico

- **Backend:** NestJS (Node.js) — 35+ endpoints API REST documentados con Swagger
- **Frontend:** React + OpenLayers — mapa interactivo con capas
- **IA:** FastAPI + scikit-learn — predicción incendio/inundación + forecast 72h
- **Base de datos:** PostgreSQL 16 + PostGIS — soporte geoespacial nativo
- **Monitoreo:** Grafana + Prometheus + Loki — 18 paneles + 7 reglas de alerting

---

Costos operativos estimados

Concepto	Costo mensual
Servidor Debian (existente PNN)	$0 (ya asignado)
Electricidad + red	Incluido en presupuesto TI
NASA FIRMS API	$0 (gratuito)
OroraTech API (contratado)	Según plan comercial
IDEAM datos abiertos	$0 (datos.gov.co)

Concepto	Costo mensual
Soporte técnico BC Fabric	Por definir
Licencias de software	**$0**

Propuesta de servicios BC Fabric SAS

Fase 1 — Despliegue (completada)

- Instalación on-premise en servidor PNN
- Configuración de red, SSL, Docker
- Integración con 8 fuentes de datos
- Capacitación inicial a operadores

Fase 2 — Soporte y evolución (propuesta)

- Soporte técnico remoto L1/L2
- Actualizaciones de seguridad mensuales
- Nuevas integraciones bajo demanda (sensores IoT, drones)
- Reentrenamiento de modelos IA con datos históricos
- Dashboard de reportes personalizados

Fase 3 — Escalamiento (futuro)

- Réplica a otras áreas protegidas (SINAP completo)
- Integración MODIS/VIIRS NRT (near-real-time por correo NASA)
- App móvil offline para guardaparques
- API pública para interoperabilidad con SIAC, RUNAP

Contacto

BC Fabric SAS Fernando Bolívar fbolivarb@gmail.com

Documento generado: Abril 2026 Versión del sistema: Manobi Sentinel v0.4

Arquitectura — Manobi Sentinel

Diagrama de arquitectura (Mermaid)

> GitHub renderiza este bloque como un diagrama interactivo. Para exportarlo como PNG: copia el bloque en https://mermaid.live y descarga.

```
graph TB
    User[Usuarios / Operadores<br/>PNN Colombia]

    subgraph EdgeLayer["Edge (HTTPS 443)"]
        Nginx[Nginx<br/>reverse proxy + TLS 1.2/1.3<br/>rate-limit + security
headers]
    end

    subgraph App["Aplicacion"]
        Frontend[Frontend React + OpenLayers<br/>vendor-ol split / lazy routes]
        API[API NestJS<br/>JWT auth + RBAC<br/>35+ endpoints]
        AI[AI Service FastAPI<br/>scikit-learn<br/>incendio + inundacion]
    end

    subgraph Geo["Servidores geoespaciales"]
        GeoServer[GeoServer<br/>WMS/WFS]
        PgTileserv[pg_tileserv<br/>MVT vectoriales]
        TileserverGL[TileServer GL<br/>tiles base offline]
    end

    subgraph DataLayer["Datos y estado"]
        Postgres[(PostgreSQL 16<br/>+ PostGIS 3.4<br/>73 parques)]
        Redis[(Redis<br/>cache + pub/sub<br/>+ BullMQ)]
        Minio[(MinIO<br/>reportes PDF/XLSX)]
    end

    subgraph Ext["Fuentes externas"]
        IDEAM[IDEAM SODA API<br/>5 datasets climaticos]
        NASA[NASA FIRMS<br/>VIIRS + MODIS hotspots]
    end

    subgraph Notif["Notificaciones"]
        Postfix[Postfix<br/>relay Gmail]
        Gmail[Gmail SMTP relay<br/>smtp-relay.gmail.com:587]
        Push[Web Push<br/>VAPID]
        Webhook[Webhooks HTTP]
    end

    subgraph Obs["Observabilidad"]
        Prom[Prometheus]
        Graf[Grafana<br/>18 paneles + 7 alertas]
        Loki[Loki + Promtail]
    end

    User -->|HTTPS| Nginx
    Nginx --> Frontend
    Nginx --> API
    Nginx --> GeoServer
    Nginx --> PgTileserv
    Nginx --> TileserverGL
    Nginx --> Graf

    Frontend -.->|REST + WS| API
    API --> Postgres
    API --> Redis
```

```
    API --> Minio
    API --> AI
    AI --> Postgres

    API -->|cron 15 min| IDEAM
    API -->|cron 30 min| NASA

    API -->|publish alerta| Redis
    Redis -->|subscribe| API
    API --> Postfix
    Postfix --> Gmail
    API --> Push
    API --> Webhook

    Postgres -.->|exporter| Prom
    Redis -.->|exporter| Prom
    API -.->|/metrics| Prom
    Prom --> Graf
    API -.->|logs| Loki
    Loki --> Graf

    classDef user fill:#2d3748,stroke:#4a5568,color:#fff
    classDef edge fill:#2b6cb0,stroke:#2c5282,color:#fff
    classDef app fill:#2f855a,stroke:#276749,color:#fff
    classDef data fill:#b7791f,stroke:#975a16,color:#fff
    classDef ext fill:#702459,stroke:#521b41,color:#fff
    classDef obs fill:#4a5568,stroke:#2d3748,color:#fff

    class User user
    class Nginx edge
    class Frontend,API,AI app
    class Postgres,Redis,Minio data
    class IDEAM,NASA ext
    class Prom,Graf,Loki obs
```

Vista general (ASCII fallback)

```
                        +-----------------------+
                        |  Usuarios / Operadores |
                        +----------+------------+
                                   | HTTPS 443
                                   v
                             +---------+
                             |  Nginx  |  reverse proxy + SSL + rate limit + headers
                             +----+----+
        +----------+-------------+--------------+------------+-----------+
        v          v             v              v            v           v
    Frontend   API (NestJS)  GeoServer   pg_tileserv   TileServer GL  Grafana
     (React)        |        (WMS/WFS)    (MVT)        (tiles base)
                    |
       +------------+---------------------+
       v            v                     v
    PostgreSQL   Redis                 MinIO        AI Service (FastAPI)
    + PostGIS    (cache + queue)       (S3)         scikit-learn
```

Redes Docker

- **frontend-net** — nginx ↔ frontend
- **backend-net** — api, postgres, redis, minio, ai-service, geo-services, postfix
- **monitoring-net** — prometheus, grafana, loki, promtail

Decisiones clave

Tema	Elección	Motivo
Mapa frontend	OpenLayers + tiles locales	Sin API keys externas, cumple requisito on-premise
Base de datos	PostgreSQL 16 + PostGIS 3.4	Estándar GIS, soporte JSONB para reglas dinámicas
Colas	BullMQ sobre Redis	Misma infra que cache, menos servicios
Observabilidad	Prometheus + Grafana + Loki	Stack abierto, sin licencias
SSL	Nginx con TLS 1.2/1.3	Obligatorio por requerimiento gubernamental
IA	scikit-learn + árbol decisión	Interpretabilidad exigida por entidad
Reportes PDF		HTML→PDF fiel, misma plantilla que el frontend

Tema	Elección	Motivo
	Puppeteer + Chromium headless	

Flujo de alerta (Mermaid)

```
sequenceDiagram
    autonumber
    participant IDEAM as IDEAM SODA
    participant NASA as NASA FIRMS
    participant DB as PostgreSQL
    participant Engine as Alert Engine<br/>(cron 15 min)
    participant AI as AI Service
    participant Redis as Redis pub/sub
    participant Listener as Alert Listener
    participant SMTP as Postfix + Gmail
    participant User as fbolivarb@gmail.com

    Note over IDEAM,DB: Ingesta automatica
    IDEAM->>DB: INSERT eventos (cron 15 min)
    NASA->>DB: INSERT hotspots (cron 30 min)

    Note over Engine,AI: Evaluacion de reglas
    Engine->>DB: SELECT reglas activas + parques
    loop por cada parque (73)
        Engine->>DB: contextoPorParque (con cache 20 min)
        Engine->>AI: predictIncendio + predictInundacion
        AI-->>Engine: probabilidad + clase
        Engine->>Engine: evaluar condicion JSONB
    end

    Note over Engine,Redis: Dedup + publish
    Engine->>DB: findOne(parque_id, tipo, activa)
    alt Alerta no existe
        Engine->>DB: INSERT alertas
        Engine->>Redis: PUBLISH manobi:alertas
    else Alerta ya existe
        Engine-->>Engine: dedup++ (no publish)
    end

    Note over Listener,User: Notificacion
    Redis->>Listener: SUBSCRIBE manobi:alertas
    Listener->>DB: SELECT suscripciones (nivel match)
    Listener->>SMTP: sendMail(subject, html)
    SMTP->>User: Email entregado (250 OK)
```

Flujo de alerta (ASCII fallback)

```
IDEAM / sensores --> eventos_climaticos (Postgres)
                        |
                        v
             Alert Engine (@Cron 15 min)
                        |
         +-------------+-------------+
         v             v             v
    Regla JSONB   Prediccion IA  Deduplicacion
                        |
                        v
                  alertas (Postgres)
                        |
         +-------------+-------------+
         v             v             v
      WebSocket    BullMQ email   Webhook
      (frontend)   (Postfix)      (http post)
```

Manual de operación — Manobi Sentinel

> Guía técnica para operar la plataforma en Debian 12 con Docker Compose. Server de referencia: `192.168.50.5`, ruta `/opt/manobi-sentinel`.

1. Tareas rutinarias

Acción	Comando
Ver estado general	`docker compose ps`
Logs en vivo	`docker compose logs -f --tail=200 <servicio>`
Reiniciar servicio	`docker compose restart <servicio>`
Reconstruir tras cambio	`docker compose up -d --build <servicio>`
Actualizar imágenes base	`docker compose pull && docker compose up -d`
Verificar backups	`ls -lh /opt/manobi/backups/`

2. Servicios y puertos internos

Servicio	Puerto interno	Rol
nginx	80/443	Reverse proxy + SSL
frontend	80 (nginx interno)	React/Vite
api	3000	NestJS
ai-service	8000	FastAPI + scikit-learn (4 workers uvicorn)
postgres	5432	PostGIS 16-3.4
redis	6379	Cache + pub/sub
minio	9000	Storage de reportes
geoserver	8080	WMS/WFS
pg_tileserv / pg_featureserv	7800 / 9000	Tiles vectoriales
tileserver-gl	8080	Tiles base OSM
grafana	3000	Dashboards (`/grafana`)
prometheus	9090	Métricas
loki / promtail	3100 / –	Logs

Servicio	Puerto interno	Rol
postgres/redis/node exporter	9187 / 9121 / 9100	Exporters Prometheus
postfix	25/587	SMTP para alertas

3. Health checks y URLs

- `https://192.168.50.5/` — dashboard operador
- `https://192.168.50.5/api/docs` — Swagger (31 endpoints)
- `https://192.168.50.5/api/health` — API health
- `https://192.168.50.5/api/metrics` — Prometheus métricas del API
- `https://192.168.50.5/grafana/` — dashboards + alerting
- `https://192.168.50.5/geoserver/web/` — GeoServer
- `https://192.168.50.5/tiles/` — TileServer GL

4. Backup y restauración

Backup manual

```
bash /opt/manobi-sentinel/database/backups/backup.sh
ls -lh /opt/manobi/backups/
```

Cron del host (ya activo)

```
crontab -l
# 0 2 * * * /opt/manobi-sentinel/database/backups/backup.sh >> /var/log/manobi-
        backup.log 2>&1
```

Retención: 30 días (dumps PostgreSQL + mirror de MinIO).

Restaurar PostgreSQL

```
docker exec -i manobi-postgres pg_restore -U manobi -d manobi --clean < \
  /opt/manobi/backups/manobi-YYYYMMDD-HHMMSS.dump
```

Restaurar MinIO (reportes)

```
docker cp /opt/manobi/backups/minio-reportes-YYYYMMDD-HHMMSS manobi-minio:/tmp/
        restore
docker exec manobi-minio mc mirror --overwrite /tmp/restore local/reportes
```

5. Cargar shapefile oficial PNN (RUNAP)

El dataset oficial viene de la API ArcGIS de Parques Nacionales Naturales.

```
# Descargar GeoJSON oficial (65 áreas protegidas)
curl -sk -m 60 -o /tmp/pnn_official.geojson \
  'https://mapas.parquesnacionales.gov.co/arcgis/rest/services/pnn/Limites_oficiales_Poligono/Fea
        where=1%3D1&outFields=ap_nombre,ap_categoria,territorial,area_calculada_ha,ap_sigla&resul

# Importar a tabla temporal con ogr2ogr
source /opt/manobi-sentinel/.env
docker run --rm --network manobi-sentinel_backend-net \
  -v /tmp/pnn_official.geojson:/data/pnn.geojson \
  ghcr.io/osgeo/gdal:alpine-small-3.9.2 \
  ogr2ogr -f PostgreSQL \
    "PG:host=postgres port=5432 dbname=$POSTGRES_DB user=$POSTGRES_USER
        password=$POSTGRES_PASSWORD" \
    /data/pnn.geojson -nln parques_import_tmp -nlt MULTIPOLYGON \
    -lco GEOMETRY_NAME=geom -overwrite -t_srs EPSG:4326

# Merge por similitud de nombre
docker exec -i manobi-postgres psql -U $POSTGRES_USER -d $POSTGRES_DB <<'SQL'
CREATE EXTENSION IF NOT EXISTS pg_trgm;
UPDATE parques p
SET geometria = t.geom,
    area_ha   = COALESCE(NULLIF(t.area_calculada_ha, 0), p.area_ha),
    region    = COALESCE(t.territorial, p.region)
FROM parques_import_tmp t
WHERE similarity(UPPER(p.nombre), UPPER(t.ap_nombre)) > 0.6;

INSERT INTO parques (nombre, geometria, region, nivel_riesgo, area_ha, descripcion)
SELECT t.ap_nombre, t.geom, t.territorial, 'medio',
       NULLIF(t.area_calculada_ha, 0), 'Importado desde RUNAP oficial'
FROM parques_import_tmp t
WHERE NOT EXISTS (
  SELECT 1 FROM parques p
  WHERE similarity(UPPER(p.nombre), UPPER(t.ap_nombre)) > 0.6
);
DROP TABLE parques_import_tmp;
SQL
```

Al cierre de 2026-04-15 la tabla tiene **73 parques** (37 con geometría oficial, 28 nuevos, 8 legacy seed pendientes de matching manual).

6. Rotar certificado SSL

```
# Autofirmado (10 años)
/opt/manobi-sentinel/scripts/rotate-ssl-cert.sh self-signed 192.168.50.5 192.168.50.5

# Let's Encrypt (requiere dominio público + puerto 80 libre)
/opt/manobi-sentinel/scripts/rotate-ssl-cert.sh letsencrypt sentinel.pnn.gov.co
        admin@pnn.gov.co
```

Para quitar warnings del browser y habilitar Service Worker: importar `/opt/manobi-sentinel/nginx/ssl/manobi.crt` como CA confiable en cada equipo cliente.

7. Rotar secretos

1. Actualizar `.env`.
2. `docker compose up -d --force-recreate api ai-service`.
3. Revocar sesiones en Redis:

```
docker exec manobi-redis redis-cli -a $REDIS_PASSWORD FLUSHDB
```

8. IDEAM

Cambiar de simulado a real (y viceversa)

Editar `IDEAM_MODE=real|simulado` en `.env`, luego:

```
docker compose up -d --force-recreate api
```

Datasets SODA usados (modo real)

ID	Tipo
sbwg-7ju4	Temperatura ambiente (°C)
uext-mhny	Humedad del aire (%)
s54a-sgyg	Precipitación (mm)
sgfv-3yp8	Velocidad viento (m/s → km/h ×3.6)
62tk-nxj5	Presión atmosférica (hPa)

Poll cada 30 min vía cron del servicio `api`. Fallback automático a simulado si la API SODA está caída.

Poll manual a demanda

```
TOKEN=$(curl -sk -X POST https://192.168.50.5/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@manobi.local","password":"..."}' | jq -r .access_token)

curl -sk -X POST -H "Authorization: Bearer $TOKEN" https://192.168.50.5/api/ideam/
        poll
```

9. Motor de alertas

- Corre cada 15 min (`0 */15 * * * *`, zona `America/Bogota`).

- Para 73 parques y 5 reglas toma ~5-7 min. Procesa secuencialmente (`BATCH = 1` en alert-engine.service.ts) para no saturar Postgres con `ST_DWithin` concurrentes.
- **Si un ciclo se queda colgado** y saturan el Postgres:

```
# Ver queries activas
docker exec manobi-postgres psql -U manobi -d manobi -c \

        "SELECT pid, age(now(), query_start), left(query,80) FROM
        pg_stat_activity WHERE state='active' AND pid <> pg_backend_pid() ORDER
        BY query_start;"

# Matar queries ST_DWithin viejas (>30 s)
docker exec manobi-postgres psql -U manobi -d manobi -c \

        "SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE
        state='active' AND query ILIKE '%ST_DWithin%' AND age(now(), query_start)
        > interval '30 seconds';"

# Restart api para limpiar promesas pendientes
docker compose restart api
```

- `statement_timeout` global: 60 s (se aplicó vía `ALTER DATABASE manobi SET statement_timeout = 60000`). Queries pesadas se abortan solas.

Reglas

Ver/editar vía: - `GET /api/reglas` (cualquier rol autenticado) - `POST /api/reglas` (admin) - `PATCH /api/reglas/:id` (admin) - `DELETE /api/reglas/:id` (admin)

10. Modelos IA (ai-service)

- Versión actual: **v2** (reentrenados 2026-04-15 con scores centrados en climas de Colombia).
- Modelos viejos v1 se mantienen en disco para rollback: `/app/models/incendio_v1.joblib` , `/app/models/inundacion_v1.joblib` .
- **Reentrenar manual:**

```
curl -sk -X POST -H "Authorization: Bearer $TOKEN" \
  https://192.168.50.5/api/predicciones/retrain/incendio
```

- **Rollback a v1:** cambiar `VERSION = 'v1'` en training.py, rebuild + up.

11. Grafana alerting

- Dashboards: folder “Manobi Sentinel / Overview” (11 paneles PostgreSQL + Prometheus + Loki).
- Alertas provisionadas: 7 reglas (API/AI down, CPU, disco, Postgres, IDEAM, engine).
- Contact point: email → `alertas@parques.gov.co` vía postfix:25.

- **Para que los emails salgan al internet real:** configurar `SMTP_RELAY_HOST` en `.env` (SMTP de la organización) y recrear postfix.
- **Ver emails en cola:**

```
docker exec manobi-postfix mailq
```

- **Validación de alerting (drill manual):**

```
docker compose stop ai-service
# esperar 3 min -> regla "AI Service caído" debe estar en estado Alerting
# email encolado en postfix
docker compose up -d ai-service
# en ~1 min la alerta pasa a estado OK
```

12. Troubleshooting rápido

Síntoma	Primera acción
Frontend no carga	`docker compose logs nginx frontend`
Login da 504 o timeout	Engine saturado; ver sección 9, matar queries y reiniciar api
`ERR_TOO_MANY_REDIRECTS` en `/grafana/`	Revisar que `proxy_pass http://grafana_backend;` (sin `/` final) en `nginx/conf.d/manobi.conf`
Heatmap vacío	Verificar tabla `predicciones` tiene datos de las últimas 24 h; revisar `IDEAM_MODE`
Alertas repetitivas por regla IA	Subir umbral vía `PATCH /api/reglas/<uid>` con `{"condicion":{...,"valor":85}}`
PostgreSQL al 900% CPU	Ver queries colgadas (sección 9)
Grafana paneles vacíos	`docker compose logs grafana | grep -i provision` — validar UID datasource coincide con dashboard

13. Checklist post-instalación

- [ ] `.env` con todos los `CHANGE_ME` reemplazados por secretos fuertes
- [ ] Crontab del host con backup diario configurado (`crontab -l`)
- [ ] `IDEAM_MODE=real` (o `simulado` si no hay internet)
- [ ] SMTP_RELAY_HOST configurado (para que alertas lleguen)
- [ ] Cert SSL: renovado con `rotate-ssl-cert.sh` si ha cambiado el host
- [ ] Modelos IA entrenados en el boot (`docker compose logs ai-service | grep "Modelos v2"`)
- [ ] Dashboard Grafana mostrando datos reales (todas las tarjetas con valores)
- [ ] Prometheus targets en estado `UP` (≥ 7 de 8; ai-service/api/node/postgres/redis mínimos)

Manual de usuario — Manobi Sentinel

Sistema de alerta temprana climática para PNN Colombia

Acceso al sistema

1. Abra su navegador (Chrome o Firefox) y visite: `https://192.168.50.5`
2. Ingrese su email y contraseña proporcionados por el administrador
3. Click “INICIAR SESIÓN”

> Si olvidó su contraseña, contacte al administrador del sistema.

Dashboard (pantalla principal)

Al ingresar verá 4 zonas:

Panel izquierdo — Alertas activas: - Lista de alertas en tiempo real ordenadas por gravedad - Rojo = crítico (requiere acción inmediata) - Amarillo = precaución (monitorear) - Las alertas se actualizan automáticamente sin recargar

Centro — Mapa interactivo: - Polígonos de colores = parques por nivel de riesgo - Puntos azules = estaciones meteorológicas IDEAM - Puntos rojos/naranjas = focos de calor satelitales (NASA + OroraTech) - Click en cualquier elemento para ver detalles - Use el panel de capas (esquina superior derecha) para activar/desactivar: - Base — mapa de fondo - Parques — polígonos de áreas protegidas - Eventos — estaciones meteorológicas - Heatmap IA — predicción de riesgo por inteligencia artificial - Puntos de calor — detecciones satelitales reales de incendios

Panel derecho — Métricas en tiempo real: - Contadores de alertas por nivel - Top 5 parques con más alertas - Predicciones IA recientes

Barra inferior — Timeline meteorológico: - Variables de temperatura, viento, lluvia de las últimas 24 horas

---

Histórico de alertas

1. Click “Histórico” en la barra de navegación
2. Use los filtros:
   - **Parque** — ver alertas de un parque específico
   - **Nivel** — filtrar solo rojo, amarillo o verde
   - **Desde / Hasta** — rango de fechas
3. El gráfico de barras muestra tendencia diaria
4. La tabla inferior lista cada alerta con detalles
5. Click **CSV** para descargar un archivo con las alertas filtradas

---

Reportes

1. Click “Reportes” en la barra de navegación
2. En el panel izquierdo:
   - Elija **tipo** de reporte (ej: “Resumen de alertas”)
   - Seleccione **formato**: PDF, Excel o CSV
   - Marque los **niveles** a incluir
   - Opcionalmente seleccione **rango de fechas**
3. Click **GENERAR**
4. El reporte aparece en la tabla de la derecha
5. Click **Descargar** para obtener el archivo

Suscripciones de notificación

1. Click “Suscripciones” en la barra de navegación
2. Cree una suscripción:
   - **Canal**: Email, Webhook o Push (navegador)
   - **Destino**: dirección de correo o URL
   - **Parque**: específico o “Todos”
   - **Niveles**: marque rojo, amarillo y/o verde
3. Las suscripciones se pueden pausar y reactivar
4. Las alertas rojo y amarillo se envían automáticamente

Reglas de alerta (administrador)

1. Click “Reglas” en la barra de navegación
2. Vea las reglas activas del motor de alertas
3. Para editar una regla: click “Editar” → modifique el JSON de condición → “GUARDAR”
4. Para crear: click “+ NUEVA” → complete nombre, nivel, condición
5. Para pausar/reactivar: use los botones en cada regla

Campos disponibles en condiciones: - `temperatura_c` — temperatura del aire (°C) - `humedad_relativa` — humedad (%) - `viento_kmh` — velocidad del viento (km/h) - `lluvia_24h_mm` — lluvia acumulada 24h (mm) - `lluvia_1h_mm` — lluvia última hora (mm) - `dias_sin_lluvia` — días consecutivos sin lluvia - `prediccion_ia.incendio` — probabilidad IA de incendio (0-100) - `prediccion_ia.inundacion` — probabilidad IA de inundación (0-100) - `parque.nivel_riesgo` — nivel de riesgo del parque (bajo/medio/alto)

Gestión de usuarios (solo administrador)

1. Click “Usuarios” en la barra de navegación
2. Vea la tabla de usuarios existentes
3. Para crear: complete nombre, email, contraseña (≥8 chars), rol

4. Roles disponibles:
    - **Admin** — acceso total, puede crear usuarios y reglas
    - **Operador** — puede generar reportes, crear alertas manuales
    - **Consulta** — solo lectura del dashboard e histórico

Grafana (monitoreo técnico)

1. Visite `https://192.168.50.5/grafana/`
2. Ingrese con las credenciales de Grafana (consulte al admin)
3. Dashboard “Manobi Sentinel / Overview” muestra:
    - Alertas activas y tendencias
    - Eventos IDEAM por tipo
    - Probabilidad IA por tipo
    - CPU, memoria, conexiones PostgreSQL
    - Logs del motor de alertas

Preguntas frecuentes

¿Cada cuánto se actualizan las alertas? El motor evalúa cada 15 minutos. Los datos IDEAM se actualizan cada 30 min. Los satélites (NASA/OroraTech) cada 1 hora.

¿Qué significan los puntos de calor en el mapa? Son detecciones reales de temperatura anómala captadas por satélites (VIIRS, MODIS, GOES). Un punto rojo dentro de un parque genera automáticamente una alerta.

¿Puedo recibir alertas en mi celular? Sí, cree una suscripción tipo “Push” y active las notificaciones cuando el navegador lo solicite. También puede crear una suscripción tipo “Email”.

¿La app funciona sin internet? Parcialmente. Si pierde conexión, verá los últimos datos cacheados. Al recuperar conexión, se actualiza automáticamente.

Manobi Sentinel v0.4 — Parques Nacionales Naturales de Colombia BC Fabric SAS — 2026

Guía de presentación — Manobi Sentinel

Para revisión/entrega a producción (PNN Colombia)

> **Objetivo:** llegar a la revisión con un discurso corto, un demo sin fallos, y respuestas listas para las preguntas típicas.

---

1. Checklist 24 horas antes

1.1 Verifica el servidor

```
# Todo healthy, 20 contenedores arriba
ssh root@192.168.50.5 "cd /opt/manobi-sentinel && docker compose ps | grep -Ev
        'healthy|Up' | head"
# Esperado: ninguna línea (todo healthy/Up)

# SSL válido
ssh root@192.168.50.5 "openssl x509 -in /opt/manobi-sentinel/nginx/ssl/manobi.crt -
        noout -enddate"
# Esperado: notAfter > hoy + 1 año

# Último backup (automático 02:00 AM diario)
ssh root@192.168.50.5 "ls -lht /opt/manobi/backups/*.dump | head -3"
# Esperado: dump de hoy, >20 MB
```

1.2 Siembra datos frescos para el demo

```
# Fuerza un polling IDEAM y NASA FIRMS para tener eventos recientes.
# Requiere un access token de admin; cámbialo por tus credenciales.
ssh root@192.168.50.5 'TOKEN=$(curl -sk -X POST https://localhost/api/auth/login \
  -H "Content-Type: application/json" \
  -d "{\"email\":\"admin@manobi.local\",\"password\":\"<TU_PASSWORD>\"}" \
  | python3 -c "import sys,json;print(json.load(sys.stdin)[\"access_token\"])") && \
  curl -sk -X POST -H "Authorization: Bearer $TOKEN" https://localhost/api/ideam/poll
        && \
  curl -sk -X POST -H "Authorization: Bearer $TOKEN" https://localhost/api/hotspots/
        poll'
```

1.3 Prepara una alerta visible para el demo en vivo

```
# Cierra una alerta amarilla; el motor la regenerará en el próximo ciclo (≤15 min)
# y se verá el publish/email en tiempo real durante la demo.
ssh root@192.168.50.5 'docker exec manobi-postgres psql -U manobi -d manobi -c \
  "UPDATE alertas SET estado='"'"'cerrada'"'"', fecha_fin=NOW() \
   WHERE id = (SELECT id FROM alertas WHERE estado='"'"'activa'"'"' AND
        nivel='"'"'amarillo'"'"' \
              ORDER BY creado_en ASC LIMIT 1) RETURNING id, tipo;"'
```

1.4 Abre en pestañas (en el orden del demo)

1. https://192.168.50.5/login (app)
2. https://192.168.50.5:3001/ (Grafana — user: admin)
3. https://192.168.50.5/api/docs (Swagger)
4. GitHub: https://github.com/fbolivar/manobi-sentinel
5. Bandeja de tu email (`fbolivarb@gmail.com`) para mostrar el correo real

---

2. Elevator pitch (60 segundos)

> Manobi Sentinel es una plataforma **on-premise** que automatiza el monitoreo y la alerta temprana climática de los **73 parques nacionales** de Colombia. Integra **8 fuentes de datos oficiales** (IDEAM + NASA FIRMS) y **modelos de IA** que predicen incendio e inundación, evaluando reglas configurables **cada 15 minutos** y notificando por **email, push y webhook**. Corre 100% en servidores del estado, sin dependencia de la nube, con dashboards, histórico, auditoría y reportes PDF/XLSX/CSV.

Menciona tres números: - **73 parques** (100% del SINAP con geometría oficial RUNAP). - **8 fuentes de datos** integradas. - **<15 min** de detección automática.

---

3. Script de la presentación (30–40 min)

Sección A — Por qué existe (5 min)

Usa la tabla de `docs/propuesta-valor-pnn.md` sección “Capacidades vs. estado anterior”. Punto clave a subrayar: - Antes: detección manual por llamadas o reportes tardíos. - Ahora: automático cada 15 min, alerta por email antes de que el operador se entere por radio.

Sección B — Demo en vivo (15 min)

Sigue la sección **4. Demo paso a paso** más abajo.

Sección C — Arquitectura (5 min)

Muestra `docs/arquitectura.md` o el diagrama. Puntos clave: - **On-premise** — todo en el servidor de PNN, sin datos saliendo del país salvo APIs de fuente (NASA FIRMS). - **Docker Compose** — 20 servicios, fáciles de replicar en otro servidor. - **Stack abierto**: NestJS + React + PostgreSQL/PostGIS + Redis + MinIO + FastAPI/scikit-learn + Grafana. - **Sin licencias de pago**.

Sección D — Operación y entrega (10 min)

Sigue la sección **6. Qué dejas instalado**.

---

4. Demo paso a paso (cómo hacerlo en vivo)

Paso 1 — Login

- URL: `https://192.168.50.5/`
- Usa `fbolivarb@gmail.com` o `admin@manobi.local`.
- **Qué mostrar:** la app pide credenciales, usa JWT con refresh, brute-force protection (5 intentos = 15 min lock).

Paso 2 — Dashboard

- **Mapa OpenLayers** con polígonos de los 73 parques coloreados por `nivel_riesgo`.
- Toggle de capas: "Eventos" (puntos IDEAM), "Heatmap IA" (probabilidad de incendio), "Puntos de calor" (hotspots NASA agrupados en clusters).
- Click en un parque → panel con datos.
- Panel derecho: **alertas activas en vivo** (20 en este momento), con botón × para cerrar (admin/operador).

Paso 3 — Histórico

- `/historico` — filtra por parque, nivel, rango de fechas.
- Gráfico de barras de alertas por día.
- Botón **CSV** para exportar.

Paso 4 — Reglas

- `/reglas` — 10 reglas activas.
- Muestra una regla (ejemplo: "Temperatura anómala en páramo"): condición en JSON, nivel resultante, acción.
- Explica: **el operador puede añadir/editar reglas sin tocar código**.

Paso 5 — Suscripciones

- `/suscripciones` — muestra tu suscripción email.
- Crea una nueva en vivo (ej. webhook a un pastebin o otro email).
- Explica 3 canales: email, webhook, push del navegador.

Paso 6 — Email real (momento "wow")

- Tenés que haber cerrado la alerta antes (paso 1.3). En el próximo ciclo (:00, :15, :30, :45) aparece el email en tu bandeja.
- Abre el email → diseño branded PNN, link "Ver en Dashboard".
- Si te toca esperar, muestra un email previo.

Paso 7 — Reportes

- `/reportes` — genera un PDF en vivo con filtros.
- Explica: **diario automático 6 AM, semanal 7 AM lunes**, con PDF adjunto por email.

- Descarga el PDF generado.

Paso 8 — Grafana (monitoring)

- `https://192.168.50.5:3001/` — panel de salud del sistema.
- 18 paneles, 7 reglas de alerta de Grafana configuradas.
- Puntos: CPU, memoria, queries/sec, alertas generadas, ciclos del motor.

Paso 9 — Swagger

- `/api/docs` — 35+ endpoints documentados.
- Demo rápido: `/alertas` con Authorize.

Paso 10 — GitHub

- Muestra el repo: CI verde (✓ backend ✓ frontend ✓ docker-build).
- Tests: 18 backend (Jest) + 2 frontend (Vitest).

---

5. Preguntas frecuentes (y cómo responder)

"¿Qué pasa si se cae el servidor?"

- Backups diarios a las 2 AM en `/opt/manobi/backups/*.dump`, retención 30 días.
- Script `database/backups/restore.sh --test` que valida el dump en una DB temporal **sin tocar producción**.
- Docker Compose reinicia los contenedores con `restart: unless-stopped`.
- RTO estimado desde backup: < 30 min.

"¿Por qué on-premise y no en la nube?"

- Datos sensibles del estado colombiano.
- Sin dependencia de SLA de terceros ni tasas de egreso.
- Funciona en redes aisladas del gobierno (el servidor usa su propio tileserver local).

"¿Qué pasa si IDEAM o NASA se caen?"

- Todo es **pull** con timeout y fallback: si NASA no responde, se usa el último dato cacheado o el fallback climatológico.
- Nada bloquea el motor de alertas.

"¿Cómo se añade un parque nuevo?"

- API `POST /parques` con geometría GeoJSON o vía script de seed.
- El motor lo evalúa automáticamente en el siguiente ciclo.

“¿Se puede conectar con otros sistemas (SINAPRED, SIREDEC, etc.)?”

- Sí, vía **webhooks**: una suscripción con canal `webhook` envía un POST JSON con cada alerta al endpoint que se configure.
- O vía la API REST documentada en Swagger.

“¿Cuántos usuarios soporta?”

- Probado con 2 usuarios concurrentes; el stack (NestJS + PostgreSQL + Redis) escala a cientos sin cambios.
- El bottleneck realista sería el mapa renderizando 10K+ eventos — ya mitigado con LIMIT 500 y clustering.

“¿Seguridad?”

- JWT access (15 min) + refresh (7d) con rotación y revocación en Redis.
- Brute-force protection: 5 intentos → lock 15 min.
- Passwords con bcrypt cost 12.
- Roles: `admin`, `operador`, `consulta`.
- HTTPS obligatorio con cert SSL interno (10 años).
- Helmet + CORS restrictivo + ValidationPipe con whitelist.

“¿Cuánto cuesta operar?”

- Cero licencias (100% open source).
- Costos: servidor físico/VPS + electricidad + conexión NASA FIRMS (gratis).

“¿Qué falta para escalar a 1000 parques?”

- Partitioning de `eventos_climaticos` (hoy 300K filas, ya tiene retención de 30 días).
- HA de PostgreSQL (replicación).
- Horizontal scaling del API (stateless, se replica fácil).

---

6. Qué dejas instalado para el equipo receptor

6.1 Credenciales y accesos

Entrega un documento físico o PDF cifrado con: - IP del servidor, usuario SSH, clave o llave pública autorizada. - `admin@manobi.local` + password (pedir cambiarla en el primer login). - `fbolivarb@gmail.com` + password (o reasignar al email institucional). - Credenciales de Grafana (admin). - Token MinIO. - JWT secrets y SMTP password (en `.env` del servidor — documentar que está ahí).

6.2 Documentación en el repo (ya presente)

- `README.md` — instalación desde cero.
- `docs/arquitectura.md` — diagramas y componentes.

- `docs/operacion.md` — 13 secciones (start/stop, logs, backups, troubleshooting).
- `docs/manual-usuario.md` — para el operador que usa la UI.
- `docs/propuesta-valor-pnn.md` — para lectores no-técnicos.
- `docs/guia-presentacion.md` — **este archivo**.
- `docs/setup-smtp-ci.md` — configuración SMTP y CI/CD.
- `docs/guia-validacion-ux.md` — pasos de QA manual por página.

6.3 Procedimientos operativos (runbook)

Muestra en `docs/operacion.md` cómo: - Reiniciar un servicio: `docker compose restart <service>`. - Ver logs: `docker compose logs -f <service>`. - Hacer backup manual: `bash /opt/manobi-sentinel/database/backups/backup.sh`. - Restaurar: `bash /opt/manobi-sentinel/database/backups/restore.sh --test`. - Rotar cert SSL: `bash /opt/manobi-sentinel/scripts/rotate-ssl-cert.sh`.

6.4 Contactos y soporte

- Mantenedor actual: Fernando Bolívar (fbolivarb@gmail.com).
- Repositorio: https://github.com/fbolivar/manobi-sentinel.
- Ventana de soporte / handoff acordada con el comité.

---

7. Checklist final antes de entrar a la reunión

Servidor verificado (todos los contenedores healthy).
SSL vigente.
Último backup < 24h.
Alerta amarilla cerrada la noche anterior (para email en vivo).
Pestañas abiertas en el orden del demo.
Bandeja de email abierta y lista.
Proyector / screen share probado.
Respaldo offline: video grabado del demo (por si cae la red).
Propuesta-valor y este documento impresos o en PDF para entregar.
Datos clave memorizados: **73 parques, 8 fuentes, <15 min detección**.

---

8. Si algo falla durante el demo

Problema	Plan B
La red se cae	Muestra el video del demo pregrabado.
Frontend lento	Refresca con Ctrl+Shift+R. Si persiste, muestra Grafana para evidenciar que el API responde.
No llega el email	

Problema	Plan B
	Muestra el log del API (`docker compose logs api --tail 50`) con “Email enviado messageId=…” — evidencia técnica.
Motor de alertas no genera nuevas	Demuéstralo cerrando una alerta desde el botón × en la UI; explica que en el próximo ciclo (≤15 min) se regenera.
Login falla	Usa el otro usuario. Si ambos fallan, muestra la API directamente con curl + Swagger.

Última actualización: 2026-04-17

Configuración SMTP y CI/CD

SMTP Relay (#1)

Los emails de alertas se generan pero quedan en cola de Postfix. Para entregar al correo real:

Opción A — Relay con servidor SMTP corporativo

```
# 1. Editar .env
SMTP_RELAY_HOST=smtp.parques.gov.co     # servidor SMTP del MEN o PNN
SMTP_RELAY_PORT=587                      # típico TLS
SMTP_RELAY_USER=manobi@parques.gov.co    # si requiere autenticación
SMTP_RELAY_PASSWORD=xxxxx

# 2. Recrear postfix
docker compose up -d --force-recreate postfix

# 3. Probar
docker exec manobi-postfix sendmail -v test@parques.gov.co <<< "Subject: Test
        Manobi\n\nHola"
docker exec manobi-postfix mailq
```

Opción B — Gmail como relay (para pruebas)

```
SMTP_RELAY_HOST=smtp.gmail.com
SMTP_RELAY_PORT=587
SMTP_RELAY_USER=cuenta@gmail.com
SMTP_RELAY_PASSWORD=app-password-de-16-chars   # generar en myaccount.google.com/
        apppasswords
```

Verificar que las alertas salgan

```
docker compose stop ai-service      # dispara alerta
sleep 180                           # esperar 3 min
docker exec manobi-postfix mailq    # debe mostrar email entregado (no encolado)
docker compose up -d ai-service     # restaurar
```

CI/CD con GitHub (#3)

Crear repositorio

```
cd /opt/manobi-sentinel
git init
git add -A
git commit -m "Initial commit — Manobi Sentinel v0.4"

# Crear repo en GitHub (requiere gh CLI o crear desde web)
gh repo create BC-FABRIC-SAS/manobi-sentinel --private --source=. --push
```

Verificar pipeline

El archivo `.github/workflows/ci.yml` tiene 3 jobs: - **backend** — `npm install && npm test` (jest, 18 tests) - **frontend** — `npm install && npm run build && npm test` (vitest, 2 tests) - **docker-build** — construye 3 imágenes (api, frontend, ai-service) con buildx cache

Push a `main` o PR dispara la pipeline automáticamente.

Deploy automático (opcional)

Agregar job en `ci.yml` que haga SSH al server:

```
  deploy:
    runs-on: ubuntu-latest
    needs: [docker-build]
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.SERVER_HOST }}
          username: root
          key: ${{ secrets.SSH_KEY }}
          script: |
            cd /opt/manobi-sentinel
            git pull
            docker compose build
            docker compose up -d
```

Configurar secrets `SERVER_HOST` y `SSH_KEY` en GitHub Settings > Secrets.

Guía de validación UX — Manobi Sentinel

> Para realizar con 1 operador de PNN. Duración estimada: 45 min.

Requisitos

- Navegador Chrome/Firefox actualizado
- Acceso a `https://192.168.50.5` (LAN del servidor)
- Credenciales: las proporcionadas por el admin

Flujo de validación

1. Login (2 min)

Acceder a `https://192.168.50.5` → redirige a `/login`
Ingresar email + contraseña → Dashboard carga en < 5 s
Intentar contraseña incorrecta → mensaje "Error de autenticación"

2. Dashboard y mapa (10 min)

- Panel izquierdo: "Alertas Activas" muestra lista con nivel rojo/amarillo
- Mapa: polígonos de parques visibles con colores según nivel de riesgo
- Click sobre un parque → tooltip con nombre, región, nivel
- Checkbox "Eventos" → puntos azules de estaciones IDEAM
- Checkbox "Heatmap IA" → blobs de color sobre los parques
- Selector "Incendio" / "Inundación" en el Heatmap cambia la visualización
- Panel derecho: "Métricas en tiempo real" con contadores
- Barra inferior: timeline con datos de temperatura/viento/lluvia
- **Observaciones:** _____

3. Histórico (5 min)

- Click "Histórico" en la barra de navegación
- Filtrar por parque → la tabla y el gráfico de barras se actualizan
- Filtrar por nivel "rojo" → solo aparecen alertas rojas
- Seleccionar rango de fechas → datos coherentes
- Click "CSV" → se descarga archivo con las alertas filtradas
- **Observaciones:** _____

4. Reportes (5 min)

- Click "Reportes" → formulario de generación a la izquierda
- Generar reporte PDF → mensaje "Reporte generado correctamente"
- El reporte aparece en la tabla de la derecha

- Click “Descargar” → se abre el PDF en nueva pestaña
- Probar con XLSX y CSV
- **Observaciones:** _____

5. Suscripciones (5 min)

- Click “Suscripciones” → formulario canal Email
- Crear suscripción con nivel rojo + parque específico
- La suscripción aparece en la lista con indicador verde pulsante
- Pausar la suscripción → indicador gris
- Reactivar → indicador verde otra vez
- **Observaciones:** _____

6. Reglas de alerta (5 min, solo admin)

- Click “Reglas” → lista de reglas con indicador activa/pausada
- Click “Editar” en una regla → panel lateral con JSON de condición
- Modificar el valor del umbral → “GUARDAR” → regla actualizada
- Click “+ NUEVA” → crear regla de prueba → aparece en la lista
- Eliminar la regla de prueba → desaparece
- **Observaciones:** _____

7. Gestión de usuarios (5 min, solo admin)

- Click “Usuarios” → tabla con usuarios existentes
- Crear usuario operador con email/password válidos → aparece en tabla
- Cambiar rol a “consulta” desde el dropdown → cambio instantáneo
- Desactivar el usuario → botón cambia a “Activar”
- Eliminar el usuario de prueba
- **Observaciones:** _____

8. Grafana (5 min)

- Acceder a `https://192.168.50.5/grafana/` → login de Grafana
- Dashboard “Manobi Sentinel / Overview” → paneles con datos reales
- Stat “Alertas activas — ROJO” tiene un valor numérico
- Gráfico “Alertas por hora” muestra barras
- Gráfico “Eventos IDEAM por tipo” muestra curvas de los 5 tipos
- Panel “Logs” muestra entradas recientes del motor
- **Observaciones:** _____

Feedback del operador

¿Qué fue fácil de entender?

¿Qué fue confuso o difícil de encontrar?

¿Qué información adicional le gustaría ver en el dashboard?

¿Qué acciones le gustaría poder hacer que hoy no puede?

Calificación general (1-5): ___